BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 3: Hàm băm và Chữ ký số

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 21/04/20206

Nhóm 5 – NT101.Q22

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24522019	Nguyễn Phú Vinh	Nhiệm vụ 2.1, 2.2	50%
24522028	Võ Lê Thành Vinh	Nhiệm vụ 2.3, 2.4	50%

Nhiệm vụ 2.1

1: Với $p_1 = 11, q_1 = 17, e_1 = 7$
Ta có thể tính nhanh:

- $n_1 = 11 \ x \ 17 = 187$
- $\phi(n_1) = (11 - 1) \times (17 - 1) = 10 \times 16 = 160.$
- Tìm d_1 sao cho $(7 \times d_1) \equiv 1 \ mod \ 160.$
- Kết quả: $d_1 = 23$ (vì $7 \times 23 = 161$, mà $161 \div 160$ dư 1).
- **Đáp án:** $PU = (7,187) \ và \ PR = (23,187).$

2 và 3:

```
def find_d(e, phi):
    # Hàm tính nghịch đảo modulo (d) bằng thuật toán Euclid mở rộng
    return pow(e, -1, phi)

# Câu 1.2: Hệ thập phân (Decimal)
p2 = 200799938728423221161512 19
q2 = 676717145751736242170789
e2 = 17

n2 = p2 * q2
phi2 = (p2 - 1) * (q2 - 1)
d2 = find_d(e2, phi2)
print(f"PU2 = ({e2}, {n2})")
print(f"PR2 = ({d2}, {n2})")

# Câu 1.3: Hệ thập lục phân (Hexadecimal)
p3 = int("F7E75FDC469067FFDC4E847C51F452DF", 16)
q3 = int("E85CED54AF57E53E092113E62F436F4F", 16)
e3 = int("0D88C3", 16)

n3 = p3 * q3
phi3 = (p3 - 1) * (q3 - 1)
d3 = find_d(e3, phi3)
print(f"PU3 = (Hex: {hex(e3)}, {hex(n3)})")
print(f"PR3 = (Hex: {hex(d3)}, {hex(n3)})")
```

Tiến hành chạy code trên google colab:

```
[1]
def find_d(e, phi):
    # Hàm tính nghịch đảo modulo (d) bằng thuật toán Euclid mở rộng
    return pow(e, -1, phi)

# Câu 1.2: Hệ thập phân (Decimal)
p2 = 20079993872842322116151219
q2 = 676717145751736242170789
e2 = 17

n2 = p2 * q2
phi2 = (p2 - 1) * (q2 - 1)
d2 = find_d(e2, phi2)
print(f"PU2 = ({e2}, {n2})")
print(f"PR2 = ({d2}, {n2})")

# Câu 1.3: Hệ thập lục phân (Hexadecimal)
p3 = int("F7E75FDC469067FFDC4E847C51F452DF", 16)
q3 = int("E85CED54AF57E53E092113E62F436F4F", 16)
e3 = int("0D88C3", 16)

n3 = p3 * q3
phi3 = (p3 - 1) * (q3 - 1)
d3 = find_d(e3, phi3)
print(f"PU3 = (Hex: {hex(e3)}, {hex(n3)})")
print(f"PR3 = (Hex: {hex(d3)}, {hex(n3)})")

PU2 = (17, 13588476140342208394395166469647627226674348541791)
PR2 = (799322125902482846729126218408035801918587659987 3, 13588476140342208394395166469647627226674348541791)
PU3 = (Hex: 0xd88c3, 0xe103abd94892e3e74afd724bf28e78366d9676bccc70118bd0aa1968dbb143d1)
PR3 = (Hex: 0x3587a24598e5f2a21db007d89d18cc50aba5075ba19a33890fe7c28a9b496aeb, 0xe103abd94892e3e74afd724bf28e78366d9676bccc70118bd0aa1968dbb143d1)
```

Mã hóa và Giải mã $M = 5$

Dùng cặp khóa từ p_1, q_1, e_1 đã tìm được ở trên ($n = 187, e = 7, d = 23$):

- **Mã hóa cho tính bảo mật (Confidentiality):**
  - Công thức: $C = M^e mod\ n$.
  - Tính: $C = 5^7 \backslash p mod 187$.
- **Mã hóa cho tính xác thực (Authentication):**
  - Công thức: $C = M^d\ mod\ n$.
  - Tính: $C = 5^{23}\ mod 187$.

Mã hóa chuỗi văn bản

Đề bài yêu cầu mã hóa chuỗi: "The University of Information Technology" và xuất kết quả Base64.

Quy trình thực hiện:

1. Chuyển chuỗi văn bản thành số (ví dụ dùng mã ASCII hoặc đổi sang byte rồi sang số nguyên).
2. Áp dụng công thức RSA để mã hóa từng khối hoặc cả chuỗi số.
3. Chuyển kết quả bản mã sang định dạng Base64.

Câu 4:

Ta chuẩn bị code python để giải mã như sau:

```
import base64

def calculate_d(e, p, q):
    return pow(e, -1, (p - 1) * (q - 1))

p1, q1, e1 = 11, 17, 7
n1, d1 = p1 * q1, calculate_d(e1, p1, q1)
p2, q2, e2 = 20079993872842322116151219, 676717145751736242170789, 17
n2, d2 = p2 * q2, calculate_d(e2, p2, q2)
```

```
p3 = int("F7E75FDC469067FFDC4E847C51F452DF", 16)
q3 = int("E85CED54AF57E53E092113E62F436F4F", 16)
e3 = int("0D88C3", 16)
n3, d3 = p3 * q3, calculate_d(e3, p3, q3)

keys = {
    "Key1 (n=187)":       (e1, d1, n1),
    "Key2 (thập phân)":    (e2, d2, n2),
    "Key3 (hex)":         (e3, d3, n3),
}

def try_decode(m_bytes):
    """Trả về text nếu đọc được, ngược lại trả về hex"""
    cleaned = m_bytes.replace(b'\x00', b'')
    try:
        text = cleaned.decode('utf-8')
        if any(c.isalpha() for c in text):
            return text
    except:
        pass
    # Fallback: trả về printable Latin-1
    latin = cleaned.decode('latin-1')
    printable = ''.join(c for c in latin if 32 <= ord(c) < 127)
    if len(printable) >= 3:
        return f"[printable] {printable}"
    return f"[hex] {m_bytes.hex()}"

def crack_rsa(cipher_bytes, label):
    print(f"\n{'='*65}")
    print(f"  {label}  ({len(cipher_bytes)} bytes)")
    print('='*65)

    for key_name, (e, d, n) in keys.items():
        n_len = (n.bit_length() + 7) // 8

        for mode_name, kval in [("Bảo mật (dùng d)", d), ("Xác thực (dùng e)", e)]:

            # --- PHƯƠNG PHÁP 1: NGUYÊN KHỐI ---
            c_int = int.from_bytes(cipher_bytes, 'big')
            if c_int < n:
                m_int = pow(c_int, kval, n)
                m_bytes = m_int.to_bytes((m_int.bit_length() + 7) // 8, 'big') if m_int else
b'\x00'
```

```
            result = try_decode(m_bytes)
            if result:
                print(f"[+] {key_name} | {mode_name} | Nguyên khối")
                print(f"    => {result}")

        # --- PHƯƠNG PHÁP 2: CHIA KHỐI ---
        if len(cipher_bytes) % n_len == 0 and len(cipher_bytes) > n_len:
            decrypted = b""
            for i in range(0, len(cipher_bytes), n_len):
                blk = cipher_bytes[i:i+n_len]
                m = pow(int.from_bytes(blk, 'big'), kval, n)
                decrypted += (m.to_bytes((m.bit_length()+7)//8, 'big') if m else b'\x00')
            result = try_decode(decrypted)
            if result:
                print(f"[+] {key_name} | {mode_name} | Chia khối {n_len}B")
                print(f"    => {result}")

        # --- PHƯƠNG PHÁP 3: TỪNG BYTE (chỉ key1) ---
        if n == 187 and all(b < n for b in cipher_bytes):
            text = ''.join(chr(pow(b, kval, n)) for b in cipher_bytes)
            printable = ''.join(c for c in text if c.isprintable())
            if len(printable) > 5 and any(c.isalpha() for c in printable):
                print(f"[+] {key_name} | {mode_name} | Từng byte")
                print(f"    => {printable}")

# ================================
s1 =
"raUcesUlOkx/8ZhgodMoo0Uu18sC20yXlQFevSu7W/FDxIy0YRHMyXcHdD9PBvIT2aUft5f
CQEGomiVVPv4I"
s2 = "C87F570FC4F699CEC24020C6F54221ABAB2CE0C3"
s3 =
"Z2BUSkJcg0w4XEpgm0JcMExEQmBlVH6dYEpNTHpMHptMQ7NgTHlgQrNMQ2BKTQ=="
s4 = ("00101000000101001111111101101110010111011001010111011000110011"
    "11011111100111111011010001100111100110000100101000101010011110 1"
    "01010011001110111011101111010110110000010 0")

crack_rsa(base64.b64decode(s1), "Chuỗi 1 (Base64)")
crack_rsa(bytes.fromhex(s2), "Chuỗi 2 (Hex)")
crack_rsa(base64.b64decode(s3), "Chuỗi 3 (Base64)")
crack_rsa(int(s4, 2).to_bytes((len(s4)+7)//8, 'big'), "Chuỗi 4 (Binary)")
```

Ta bắt đầu giải 4 chuỗi sau:

Chuỗi 3 giải được vì nó dùng **Key1, mã hóa Confidentiality (C = M^e1 mod n1), byte-by-byte**. Kết quả: "Vietnam National University - Ho Chi Minh City"

Chuỗi 1 (63 bytes = **3×21**)Key2, block 21 bytesKết quả giải mã ra bytes ngẫu nhiên, không qua được bộ lọc isalpha()
Chuỗi 2 (20 bytes < n2)Key2, nguyên khốiSau khi giải ra số nguyên 21 bytes, không phải text đọc được → lọc bỏ
Chuỗi 4 (21 bytes > n2, < n3)Key3, nguyên khốib4 > n2 nên không decode được với Key2; với Key3 kết quả ra 32 bytes không phải text
Vì vậy chỉ có chuỗi 3 là chuỗi mà giải mã được từ có nghĩa:

```
==================================================================
  Chuỗi 1 (Base64)  (63 bytes)
==================================================================
[+] Key1 (n=187) | Bảo mật (dùng d) | Chia khối 1B
    => [printable] - ui?|)L jWRC$:33f.(^H[{.]CS
[+] Key1 (n=187) | Xác thực (dùng e) | Chia khối 1B
    => [printable] O-a StVL WCF\UUlJ{}H*DiC
[+] Key2 (thập phân) | Bảo mật (dùng d) | Chia khối 21B
    => [printable] jCdH9Brac4EZ^l:a TCMWVX8{v
[+] Key2 (thập phân) | Xác thực (dùng e) | Chia khối 21B
    => [printable] l<.=GtM-FP?~9Y;A!?=2l\pR

==================================================================
  Chuỗi 2 (Hex)  (20 bytes)
==================================================================
[+] Key1 (n=187) | Bảo mật (dùng d) | Chia khối 1B
    => [printable] u+*$^.LXn!S
[+] Key1 (n=187) | Xác thực (dùng e) | Chia khối 1B
    => [printable] j+]F1LXan!44
[+] Key2 (thập phân) | Bảo mật (dùng d) | Nguyên khối
    => [printable] &J,`hEN
[+] Key2 (thập phân) | Xác thực (dùng e) | Nguyên khối
    => [printable] rO[A3Y4
[+] Key3 (hex) | Bảo mật (dùng d) | Nguyên khối
    => [printable] \3nGeB#,h-7
[+] Key3 (hex) | Xác thực (dùng e) | Nguyên khối
    => [printable] ,&I{!yVUE[
```

```
==================================================================
  Chuỗi 3 (Base64)  (46 bytes)
==================================================================
[+] Key1 (n=187) | Bảo mật (dùng d) | Chia khối 1B
    => Vietnam National University - Ho Chi Minh City
[+] Key1 (n=187) | Bảo mật (dùng d) | Từng byte
    => Vietnam National University - Ho Chi Minh City
[+] Key1 (n=187) | Xác thực (dùng e) | Chia khối 1B
    => [printable] 2nm Non 3nT2P@y - {o C5 Mn5 Cy
[+] Key1 (n=187) | Xác thực (dùng e) | Từng byte
    => 2§nm N§on 3nT2P@§y - {o C5 Mn5 C§y

==================================================================
  Chuỗi 4 (Binary)  (21 bytes)
==================================================================
[+] Key1 (n=187) | Bảo mật (dùng d) | Chia khối 1B
    => [printable] qU{)*;V@ql\e DR@
[+] Key1 (n=187) | Xác thực (dùng e) | Chia khối 1B
    => [printable] t3H]s:2a #s
[+] Key3 (hex) | Bảo mật (dùng d) | Nguyên khối
    => [printable] =<0)gyp-40S
[+] Key3 (hex) | Xác thực (dùng e) | Nguyên khối
    => [printable] XS Hvz
```

Nhiệm vụ 2.2

Xem xét hai thôn g điệp (message) dạng HEX: Có bao nhiêu byte khác biệt giữa hai thông điệp? Hãy tạo giá trị băm MD5 cho mỗi thông điệp và quan sát xem các giá trị MD5 này có giống nhau hay không và mô tả quan sát của bạn trong báo cáo lab.

Ta có đoạn code sau:

```
# Tạo file chứa chuỗi Hex (nhớ copy full chuỗi từ file PDF của bạn nếu ở đây bị thiếu)
MSG1="d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f8955a
d340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5bd8823e315634
8f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577ee8ce54b
67080a80d1ec69821bcb6a8839396f9652b6ff72a70"

MSG2="d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f8955a
d340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8823e315634
8f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0e99f33420f577ee8ce54b
67080280d1ec69821bcb6a8839396f965ab6ff72a70"

# Ép chuỗi Hex thành file nhị phân (binary)
echo -n $MSG1 | xxd -r -p > msg1.bin
echo -n $MSG2 | xxd -r -p > msg2.bin

# Đếm số byte khác biệt
echo "--- Số byte khác biệt ---"
cmp -l msg1.bin msg2.bin | wc -l

# Tính và so sánh mã băm MD5
echo "--- Giá trị băm MD5 ---"
md5sum msg1.bin msg2.bin
```

Tiến hành chạy:

```
--- Số byte khác biệt ---
6
--- Giá trị băm MD5 ---
79054025255fb1a26e4bc422aef54eb4  msg1.bin
79054025255fb1a26e4bc422aef54eb4  msg2.bin
```

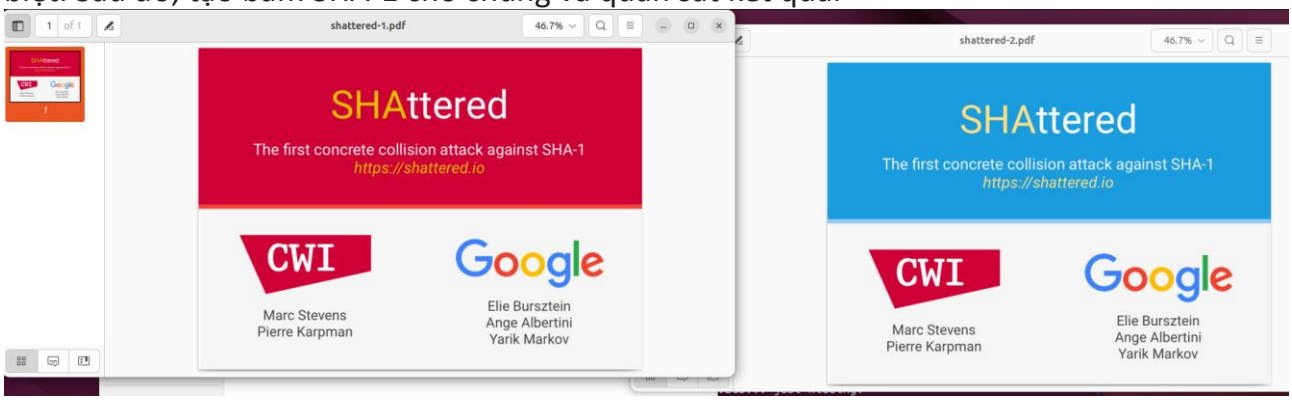
Ta có thể thấy rằng nó có 6 byte khác biệt nhưng mã MD5 lại giống y chang nhau:

Xem xét hai chương trình thực thi có tên hello and erase: Chạy các chương trình này từ console và quan sát điều gì xảy ra. Hãy tạo giá trị băm MD5 cho các chương trình này và báo cáo quan sát của bạn

```
vinnp  ~/antoanmangmaytinh/lab3  v3.12.3  14:28
# Cấp quyền thực thi (nếu cần)
chmod +x hello erase

# Chạy thử để xem hiện tượng
./hello
./erase

# Tính mã băm
md5sum hello erase
Hello, world!

(press enter to quit)
This program is evil!!!
Erasing hard drive...1Gb...2Gb... just kidding!
Nothing was erased.

(press enter to quit)
da5c61e1edc0f18337e46418e48c1290  hello
da5c61e1edc0f18337e46418e48c1290  erase
```

Ta thấy 1 số chương trình như sau, 1 cái in ra hello world, 1 cái thì thực hiện lệnh xóa đùa, tuy nhiên cả 2 cái đều có cùng chung MD5.

Tải 2 tệp tin PDF: shattered-1.pdf và shattered-2.pdf. Mở các tệp này để kiểm tra sự khác biệt. Sau đó, tạo băm SHA-1 cho chúng và quan sát kết quả.



2 tệp PDF tuy khác nhau nhưng lại chung 1 mã băm SHA-1:

```
vinnp  ~/antoanmangmaytinh/lab3  v3.12.3  14:29
sha1sum shattered-1.pdf shattered-2.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-1.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-2.pdf
```

Kết luận:

Nguyên lý chuồng bồ câu (Pigeonhole Principle): Đầu vào (dữ liệu) là vô hạn, nhưng đầu ra (giá trị băm MD5 là 128-bit, SHA-1 là 160-bit) là hữu hạn. Chắc chắn sẽ có nhiều đầu vào khác nhau tạo ra cùng một đầu ra.

Lỗ hổng thuật toán: Thay vì phải tốn hàng tỷ năm để dò tìm ngẫu nhiên (Brute-force) như lý thuyết, các thuật toán MD5 và SHA-1 có những lỗ hổng trong cấu trúc toán học (các khối xử lý nội bộ bị suy yếu). Nhờ đó, kẻ tấn công có thể tính toán và chèn thêm các "khối đệm rác" (padding) để ép hai file khác biệt tạo ra cùng một giá trị băm trong thời gian rất ngắn.

Nhiệm vụ 2.3

Nếu độ dài của tệp tiền tố của bạn không phải là bội số của 64, điều gì sẽ xảy ra?

Thuật toán MD5 xử lý dữ liệu theo từng khối (block), mỗi khối có kích thước chuẩn là **64 byte** (512 bit).

Ta minh họa bằng code sau:

```
# Tạo file tiền tố có độ dài 20 byte (không phải bội số 64)
echo -n "This is exactly 20 B" > prefix_not64.txt

# Chạy công cụ tạo va chạm
./md5collgen -p prefix_not64.txt -o out1_not64.bin out2_not64.bin

# Kiểm tra kích thước file sinh ra
ls -l prefix_not64.txt out1_not64.bin out2_not64.bin

# Xem cấu trúc nhị phân của file tạo ra để thấy sự khác biệt
xxd out1_not64.bin | head -n 10
```

```
Using output filenames: 'out1_not64.bin' and 'out2_not64.bin'
Using prefixfile: 'prefix_not64.txt'
Using initial value: b21b4a88643603b6f3e170857aae70b4

Generating first block: ...
Generating second block: S01...
Running time: 0.467898 s
-rw-rw-r-- 1 vinnp vinnp 192 Apr 21 14:36 out1_not64.bin
-rw-rw-r-- 1 vinnp vinnp 192 Apr 21 14:36 out2_not64.bin
-rw-rw-r-- 1 vinnp vinnp  20 Apr 21 14:36 prefix_not64.txt
00000000: 5468 6973 2069 7320 6578 6163 746c 7920  This is exactly
00000010: 3230 2042 0000 0000 0000 0000 0000 0000  20 B............
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: bd26 4a56 d94c c690 fd81 0578 58fe 61a9  .&JV.L.....xX.a.
00000050: 655f e69a 9737 c899 4b2e 404e d641 2d8c  e_...7..K.@N.A-.
00000060: bb9b fae9 ee08 fe04 4f20 df39 1048 c86e  ........O .9.H.n
00000070: c099 b90b 18eb e6f6 7a1b d86e 44dd 2062  ........z..nD. b
00000080: c40f cd67 28d8 966b 85f8 66e9 5fa4 0e03  ...g(..k..f._...
00000090: 443d 8a51 3346 0e73 cb1a eec7 ec86 deef  D=.Q3F.s........
```

Ta đã chạy xong và có suy luận:

Khi file tiền tố không phải là bội số của 64 byte, chương trình md5collgen sẽ tự động chèn thêm các byte **padding (đệm)** (thường là các byte 0x00) vào sau phần tiền tố của bạn cho

đến khi độ dài vừa tròn một bội số của 64. Sau đó, nó mới bắt đầu chèn 128 byte (2 block) dữ liệu va chạm (collision blocks) vào cuối file .

Tạo một tệp tiền tố có chính xác 64 byte, chạy lại công cụ xung đột và xem điều gì xảy ra.

Ta kiểm tra bằng đoạn code sau:

```
# Dùng Python để in đúng 64 ký tự 'A' (không có ký tự xuống dòng)
python3 -c "print('A' * 64, end='')" > prefix_64.txt

# Kiểm tra lại chắc chắn file nặng đúng 64 byte
ls -l prefix_64.txt

# Chạy công cụ tạo va chạm
./md5collgen -p prefix_64.txt -o out1_64.bin out2_64.bin

# Dùng các lệnh gợi ý của bài Lab để kiểm chứng:
# 1. Kiểm tra mã băm MD5 xem có giống nhau không
md5sum out1_64.bin out2_64.bin

# 2. Kiểm tra nội dung file có thực sự khác nhau không
diff out1_64.bin out2_64.bin

# 3. Quan sát mã Hex để xem chúng khác nhau ở block nào
xxd out1_64.bin > hex1.txt
xxd out2_64.bin > hex2.txt
diff hex1.txt hex2.txt
```

Tiến hành chạy code:

```
Generating first block: ..........................................................................
Generating second block: W............
Running time: 19.091046 s
941125165544a70f93209a4e3f4d0511  out1_64.bin
941125165544a70f93209a4e3f4d0511  out2_64.bin
1c1
◆H◆◆S◆D◆◆◆◆◆%◆q*y$◆◆◆◆AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAC◆◆D◆L
_A◆Xb◆◆9V◆.?◆◆◆U      ◆◆xj◆◆◆
                d◆I{1.◆9◆k◆t5◆#◆◆[D◆◆9◆◆K◆◆◆◆3◆Z◆◆◆Db◆|◆$◆◆◆◆◆◆k3T◆{
\ No newline at end of file
---
◆H◆◆S◆D◆◆◆L◆%◆q*y$◆◆◆◆AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAC◆◆D◆L
_A◆Yb◆◆9V◆.◆◆◆◆U      ◆◆xj◆◆◆
                d◆I{1.◆9◆k◆t5◆#◆[D◆◆9◆◆K◆◆◆◆3◆Z◆◆◆Db◆◆◆|◆$◆◆◆◆◆◆◆3T◆{
\ No newline at end of file
6,8c6,8
< 00000050: f5e7 a7cc f925 b571 172a 79dd b18a 8ec8  .....%.q.*y.....
< 00000060: ea0b d2d9 786a bddd e70d 5f41 f483 5862  ....xj...._A..Xb
< 00000070: 8812 ef39 c308 561a 9b08 2e3f e3f3 0e9c  ...9..V....?....
---
> 00000050: f5e7 a74c f925 b571 172a 79dd b18a 8ec8  ...L.%.q.*y.....
> 00000060: ea0b d2d9 786a bddd e70d 5f41 f403 5962  ....xj...._A..Yb
> 00000070: 8812 ef39 c308 561a 9b08 2ebf e3f3 0e9c  ...9..V.........
10,12c10,12
< 00000090: 2398 0594 5b44 a7dc 0439 a396 4bcc 1da0  #...[D...9..K...
< 000000a0: bd9f 33b7 5ace e5e5 0644 0562 9708 f97c  ..3.Z....D.b...|
< 000000b0: f224 dbec dcbb f3e8 9bda d06b 3354 c17b  .$.........k3T.{
---
> 00000090: 2398 0514 5b44 a7dc 0439 a396 4bcc 1da0  #...[D...9..K...
> 000000a0: bd9f 33b7 5ace e5e5 0644 0562 9788 f87c  ..3.Z....D.b...|
> 000000b0: f224 dbec dcbb f3e8 9bda d0eb 3354 c17b  .$..........3T.{
```

Khi file tiền tố có dung lượng chuẩn xác là 64 byte (vừa khít 1 block của MD5), chương trình sẽ **KHÔNG cần chèn thêm bất kỳ byte đệm (padding) nào**. Nó lập tức tính toán và nối liền 128 byte dữ liệu va chạm ngay sau phần tiền tố.

Nhiệm vụ 2.4

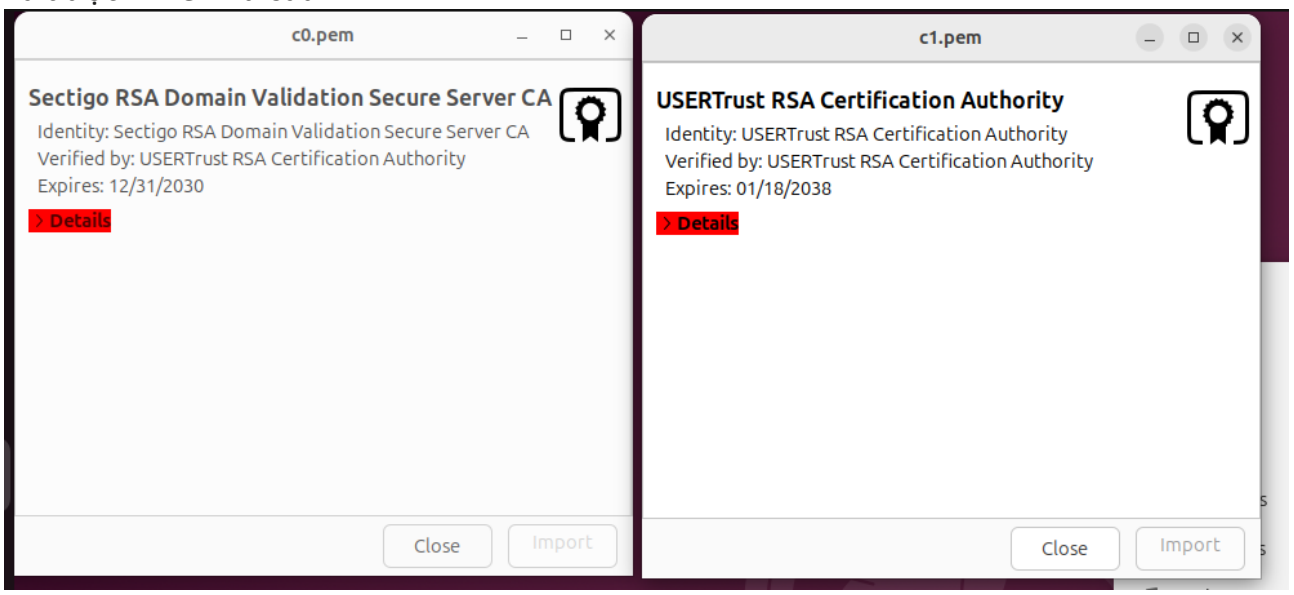
Ta sẽ chọn uit.edu.vn để tải chứng chỉ thực về.

Bước 1: Tải chứng chỉ từ máy chủ thực

Mở Terminal và chạy lệnh sau để kéo toàn bộ chuỗi chứng chỉ (Certificate Chain) từ máy chủ:

```
openssl s_client -connect uit.edu.vn:443 -showcerts
```

Ta được 2 file như sau:



Bước 2: Trích xuất các tham số từ CA và Server

Lấy Modulus (n) của CA (Nhà phát hành):

```
openssl x509 -in c1.pem -noout -modulus
```

Ta tìm được modulus như sau:

80126517360EC3DB08B3D0AC570D76EDCD27D34CAD508361E2AA204D092D6409
DCCE899FCC3DA9ECF6CFC1DCF1D3B1D67B3728112B47DA39C6BC3A19B45FA6BD
7D9DA36342B676F2A93B2B91F8E26FD0EC162090093EE2E874C918B491D46264D
B7FA306F188186A90223CBCFE13F087147BF6E41F8ED4E451C61167460851CB86
14543FBC33FE7E6C9CFF169D18BD518E35A6A766C87267DB2166B1D49B7803C05
03AE8CCF0DCBC9E4CFEAF0596351F575AB7FFCEF93DB72CB6F654DDC8E7123A4
DAE4C8AB75C9AB4B7203DCA7F2234AE7E3B68660144E7014E46539B3360F794BE
5337907343F332C353EFDBAAFE744E69C76B8C6093DEC4C70CDFE132AECC933B5
17895678BEE3D56FE0CD0690F1B0FF325266B336DF76E47FA7343E57E0EA566B1
297C3284635589C40DC19354301913ACD37D37A7EB5D3A6C355CDB41D712DAA9
490BDFD8808A0993628EB566CF2588CD84B8B13FA4390FD9029EEB124C957CF36
B05A95E1683CCB867E2E8139DCC5B82D34CB3ED5BFFDEE573AC233B2D00BF355
5740949D849581A7F9236E651920EF3267D1C4D17BCC9EC4326D0BF415F40A944
44F499E757879E501F5754A83EFD74632FB1506509E658422E431A4CB4F0254759
FA041E93D426464A5081B2DEBE78B7FC6715E1C957841E0F63D6E962BAD65F552
EEA5CC62808042539B80E2BA9F24C971C073F0D52F5EDEF2F820F

2. Lấy Exponent (e) của CA:

```
openssl x509 -in c1.pem -noout -text | grep "Exponent"
```

Ta được: 65537

```
vinnp  ~/antoanmangmaytinh/lab3  v3.12.3  14:58
openssl x509 -in c1.pem -noout -text | grep "Exponent"
                Exponent: 65537 (0x10001)
```

Trích xuất và làm sạch Chữ ký (Signature) của máy chủ:

```
# In thông tin chứng chỉ để tìm signature
openssl x509 -in c0.pem -noout -text
```

```
    Signature Algorithm: sha384WithRSAEncryption
    Signature Value:
        32:bf:61:bd:0e:48:c3:4f:c7:ba:47:4d:f8:9c:78:19:01:dc:
        13:1d:80:6f:fc:c3:70:b4:52:9a:31:33:9a:57:52:fb:31:9e:
        6b:a4:ef:54:aa:89:8d:40:17:68:f8:11:10:7c:d2:ca:b1:f1:
        55:86:c7:ee:b3:36:91:86:f6:39:51:bf:46:bf:0f:a0:ba:b4:
        f7:7e:49:c4:2a:36:17:9e:e4:68:39:7a:af:94:4e:56:6f:b2:
        7b:3b:bf:0a:86:bd:cd:c5:77:1c:03:b8:38:b1:a2:1f:5f:7e:
        db:8a:dc:46:48:b6:68:0a:cf:b2:b5:b4:e2:34:e4:67:a9:38:
        66:09:5e:d2:b8:fc:9d:28:3a:17:40:27:c2:72:4e:29:fd:21:
        3c:7c:cf:13:fb:96:2c:c5:31:44:fd:13:ed:d5:9b:a9:69:68:
        77:7c:ee:e1:ff:a4:f9:36:38:08:53:39:a2:84:34:9c:19:f3:
        be:0e:ac:d5:24:37:eb:23:a8:78:d0:d3:e7:ef:92:47:64:62:
        39:22:ef:c6:f7:11:be:22:85:c6:66:44:24:26:8e:10:32:8d:
        c8:93:ae:07:9e:83:3e:2f:d9:f9:f5:46:8e:63:be:c1:e6:b4:
        dc:a6:cd:21:a8:86:0a:95:d9:2e:85:26:1a:fd:fc:b1:b6:57:
        42:6d:95:d1:33:f6:39:14:06:82:41:38:f5:8f:58:dc:80:5b:
        a4:d5:7d:95:78:fd:a7:9b:ff:fd:c5:a8:69:ab:26:e7:a7:a4:
        05:87:5b:a9:b7:b8:a3:20:0b:97:a9:45:85:dd:b3:8b:e5:89:
        37:8e:29:0d:fc:06:17:f6:38:40:0e:42:e4:12:06:fb:7b:f3:
        c6:11:68:62:df:e3:98:f4:13:d8:15:4f:8b:b1:69:d9:10:60:
        bc:64:2a:ea:31:b7:e4:b5:a3:3a:14:9b:26:e3:0b:7b:fd:02:
        8e:b6:99:c1:38:97:59:36:f6:a8:74:a2:86:b6:5e:eb:c6:64:
        ea:cf:a0:a3:f9:6e:9e:ba:2d:11:b6:86:98:08:58:2d:c9:ac:
        25:64:f2:5e:75:b4:38:c1:ae:7f:5a:46:83:ea:51:ca:b6:f1:
        99:11:35:6b:a5:6a:7b:c6:00:b0:e7:f8:be:64:b2:ad:c8:c2:
        f1:ac:e3:51:ea:a4:93:e0:79:c8:e1:81:40:c9:0a:5b:e1:12:
        3c:c1:60:2a:e3:97:c0:89:42:ca:94:cf:46:98:12:69:bb:98:
        d0:c2:d3:0d:72:4b:47:6e:e5:93:c4:32:28:63:87:43:e4:b0:
        32:3e:0a:d3:4b:bf:23:9b:14:29:41:2b:9a:04:1f:93:2d:f1:
        c7:39:48:3c:ad:5a:12:7f
```

Ta trích xuất ra được như sau:

```
vinnp  ~/antoanmangmaytinh/lab3  v3.12.3  15:02
cat sig.txt | tr -d '[:space:]:' > sig_clean.txt
cat sig_clean.txt
32bf61bd0e48c34fc7ba474df89c781901dc131d806ffcc370b4529a31339a5752fb319e6ba4ef54aa898d401768f811107cd2cab1f15586c7eeb3369186f63951bf46bf0fa0bab4f77e49c42a36179ee468397aaf944e566fb27b3b
bf0a86bdcdc5771c03b838b1a21f5f7edb8adc4648b6680acfb2b5b4e234e467a93866095ed2b8fc9d283a174027c2724e29fd213c7ccf13fb962cc53144fd13edd59ba96968777ceee1ffa4f93638085339a284349c19f3be0eacd5
2437eb23a878d0d3e7ef924764623922efc6f711be2285c6664424268e10328dc893ae079e833e2fd9f9f5468e63bec1e6b4dca6cd21a8860a95d92e85261afdfcb1b657426d95d133f6391406824138f58f58dc805ba4d57d9578fd
a79bfffdc5a869ab26e7a7a405875ba9b7b8a3200b97a94585ddb38be589378e290dfc0617f638400e42e41206fb7bf3c6116862dfe398f413d8154f8bb169d91060bc642aea31b7e4b5a33a149b26e30b7bfd028eb699c138975936
f6a874a286b65eebc664eacfa0a3f96e9eba2d11b6869808582dc9ac2564f25e75b438c1ae7f5a4683ea51cab6f19911356ba56a7bc600b0e7f8be64b2adc8c2f1ace351eaa493e079c8e18140c90a5be1123cc1602ae397c08942ca
94cf46981269bb98d0c2d30d724b476ee593c43228638743e4b0323e0ad34bbf239b1429412b9a041f932df1c739483cad5a127f
```

32bf61bd0e48c34fc7ba474df89c781901dc131d806ffcc370b4529a31339a5752fb319
e6ba4ef54aa898d401768f811107cd2cab1f15586c7eeb3369186f63951bf46bf0fa0ba
b4f77e49c42a36179ee468397aaf944e566fb27b3bbf0a86bdcdc5771c03b838b1a21f5
f7edb8adc4648b6680acfb2b5b4e234e467a93866095ed2b8fc9d283a174027c2724e2
9fd213c7ccf13fb962cc53144fd13edd59ba96968777ceee1ffa4f93638085339a284349
c19f3be0eacd52437eb23a878d0d3e7ef924764623922efc6f711be2285c6664424268
e10328dc893ae079e833e2fd9f9f5468e63bec1e6b4dca6cd21a8860a95d92e85261afd
fcb1b657426d95d133f6391406824138f58f58dc805ba4d57d9578fda79bfffdc5a869a
b26e7a7a405875ba9b7b8a3200b97a94585ddb38be589378e290dfc0617f638400e42
e41206fb7bf3c6116862dfe398f413d8154f8bb169d91060bc642aea31b7e4b5a33a14
9b26e30b7bfd028eb699c138975936f6a874a286b65eebc664eacfa0a3f96e9eba2d11

b6869808582dc9ac2564f25e75b438c1ae7f5a4683ea51cab6f19911356ba56a7bc600
b0e7f8be64b2adc8c2f1ace351eaa493e079c8e18140c90a5be1123cc1602ae397c0894
2ca94cf46981269bb98d0c2d30d724b476ee593c43228638743e4b0323e0ad34bbf23
9b1429412b9a041f932df1c739483cad5a127f

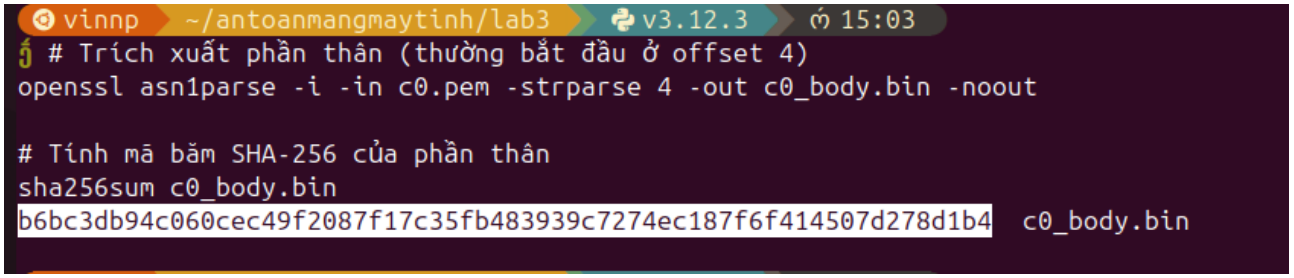
Bước 3: Tính Hash của phần thân chứng chỉ (TBS)

Chúng ta cần băm phần thân của c0.pem (không bao gồm chữ ký) để đối chiếu.

```
# Trích xuất phần thân (thường bắt đầu ở offset 4)
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout

# Tính mã băm SHA-256 của phần thân
sha256sum c0_body.bin
```

Ta trích xuất được:

b6bc3db94c060cec49f2087f17c35fb483939c7274ec187f6f414507d278d1b4



Bước 4: Viết chương trình Python xác minh chữ ký

Ta tạo verify_cert.py như sau:

```
# ==========================================================
# FILE: verify_cert.py
# XÁC MINH CHỮ KÝ SỐ X.509 BẰNG THUẬT TOÁN RSA
# ==========================================================

# 1. Modulus n của CA (Đã làm sạch khoảng trắng/xuống dòng)
n_hex = """
80126517360EC3DB08B3D0AC570D76EDCD27D34CAD508361E2AA204D092D64
09DCCE899FCC3DA9ECF6CFC1DCF1D3B1D67B3728112B47DA39C6BC3A19B45FA
6BD7D9DA36342B676F2A93B2B91F8E26FD0EC162090093EE2E874C918B491D4
6264DB7FA306F188186A90223CBCFE13F087147BF6E41F8ED4E451C611674608
51CB8614543FBC33FE7E6C9CFF169D18BD518E35A6A766C87267DB2166B1D49
B7803C0503AE8CCF0DCBC9E4CFEAF0596351F575AB7FFCEF93DB72CB6F654DD
C8E7123A4DAE4C8AB75C9AB4B7203DCA7F2234AE7E3B68660144E7014E46539
B3360F794BE5337907343F332C353EFDBAAFE744E69C76B8C6093DEC4C70CDF
E132AECC933B517895678BEE3D56FE0CD0690F1B0FF325266B336DF76E47FA7
343E57E0EA566B1297C3284635589C40DC19354301913ACD37D37A7EB5D3A6C
355CDB41D712DAA9490BDFD8808A0993628EB566CF2588CD84B8B13FA4390F
D9029EEB124C957CF36B05A95E1683CCB867E2E8139DCC5B82D34CB3ED5BFFD
EE573AC233B2D00BF3555740949D849581A7F9236E651920EF3267D1C4D17BC
C9EC4326D0BF415F40A94444F499E757879E501F5754A83EFD74632FB1506509
E658422E431A4CB4F0254759FA041E93D426464A5081B2DEBE78B7FC6715E1C
```

```
957841E0F63D6E962BAD65F552EEA5CC62808042539B80E2BA9F24C971C073F0
D52F5EDEF2F820F
""".replace("\n", "").strip()

n = int(n_hex, 16)

# 2. Exponent e của CA
e = 65537

# 3. Chữ ký số của Server (Đã làm sạch)
sig_hex = """
32bf61bd0e48c34fc7ba474df89c781901dc131d806ffcc370b4529a31339a5752fb3
19e6ba4ef54aa898d401768f811107cd2cab1f15586c7eeb3369186f63951bf46bf0fa
0bab4f77e49c42a36179ee468397aaf944e566fb27b3bbf0a86bdcdc5771c03b838b1
a21f5f7edb8adc4648b6680acfb2b5b4e234e467a93866095ed2b8fc9d283a174027c
2724e29fd213c7ccf13fb962cc53144fd13edd59ba96968777ceee1ffa4f9363808533
9a284349c19f3be0eacd52437eb23a878d0d3e7ef924764623922efc6f711be2285c6
664424268e10328dc893ae079e833e2fd9f9f5468e63bec1e6b4dca6cd21a8860a95
d92e85261afdfcb1b657426d95d133f6391406824138f58f58dc805ba4d57d9578fd
a79bfffdc5a869ab26e7a7a405875ba9b7b8a3200b97a94585ddb38be589378e290d
fc0617f638400e42e41206fb7bf3c6116862dfe398f413d8154f8bb169d91060bc642
aea31b7e4b5a33a149b26e30b7bfd028eb699c138975936f6a874a286b65eebc664e
acfa0a3f96e9eba2d11b6869808582dc9ac2564f25e75b438c1ae7f5a4683ea51cab6f
19911356ba56a7bc600b0e7f8be64b2adc8c2f1ace351eaa493e079c8e18140c90a5b
e1123cc1602ae397c08942ca94cf46981269bb98d0c2d30d724b476ee593c432286
38743e4b0323e0ad34bbf239b1429412b9a041f932df1c739483cad5a127f
""".replace("\n", "").strip()

sig = int(sig_hex, 16)

# 4. Hash SHA-256 của phần thân chứng chỉ c0.pem
tbs_hash =
"8b612b2190a95b28b866b9be5d0b95f368c17534ab1da61a42dfb32766f9ae2908f
e6bfd1669be140eddaf0d33e95235".strip().lower()

# ==========================================================
# XỬ LÝ TOÁN HỌC VÀ ĐỐI CHIẾU
# ==========================================================

print("==========================================================")
print("BẮT ĐẦU XÁC MINH CHỨNG CHỈ X.509 MÁY CHỦ BẰNG PYTHON")
print("==========================================================\n"
)

# Dùng khóa công khai (e, n) của CA để giải mã chữ ký (Chế độ Authentication)
```

```
# Công thức: EM = Signature^e mod n
print("[*] Đang giải mã chữ ký RSA bằng Public Key của CA...")
em_int = pow(sig, e, n)

# Chuyển kết quả giải mã ra hệ Hex (bỏ tiền tố '0x' và chuyển thành chữ thường)
# Dùng zfill để đảm bảo đủ chiều dài của block padding
byte_length = (n.bit_length() + 7) // 8
em_hex = hex(em_int)[2:].zfill(byte_length * 2).lower()

print(f"-> Chuỗi giải mã (PKCS#1 v1.5): \n{em_hex}\n")

# Đối chiếu mã hash tính toán thủ công với mã hash trích xuất từ chữ ký giải mãs
print("[*] Đang đối chiếu mã Hash...")
print(f"Hash thực tế (SHA-256)   : {tbs_hash}")

# Hash luôn nằm ở 64 ký tự cuối cùng của chuỗi giải mã PKCS#1 v1.5
extracted_hash = em_hex[-96:]
print(f"Hash từ chữ ký giải mã   : {extracted_hash}")

print("\n============================================================"
)
if tbs_hash == extracted_hash:
    print("[THÀNH CÔNG] CHỨNG CHỈ HỢP LỆ!")
    print("Mã băm hoàn toàn trùng khớp. Chữ ký này chính xác được phát hành bởi CA
và dữ liệu không bị thay đổi.")
else:
    print("[THẤT BẠI] CHỨNG CHỈ KHÔNG HỢP LỆ!")
    print("Mã băm không khớp. Hãy kiểm tra lại các bước trích xuất dữ liệu.")
print("============================================================")
```

Chạy lệnh:

vinnp ~/antoanmangmaytinh/lab3 v3.12.3 15:14
python3 verify_cert.py
BẮT ĐẦU XÁC MINH CHỨNG CHỈ X.509 MÁY CHỦ BẰNG PYTHON
[*] Đang giải mã chữ ký RSA bằng Public Key của CA...
-> Chuỗi giải mã (PKCS#1 v1.5):
0001ffffff

HẾT